

Concurrencia y Paralelismo

Algoritmo de Lamport

Mauricio Castillo Oloarte

Cesar E. Uc Aguilar

Uriel A. Tun Muñoz

Pedro A. Chávez Rivero

Joaquín Morales

Zabdiel Crispín

Origen del algoritmo



Edsger Dijkstra plantea formalmente el problema de la exclusión mutua entre procesos concurrentes ("problema de la sección crítica").

Leslie Lamport publica "A New Solution of Dijkstra's Concurrent Programming Problem": el algoritmo de la panadería (bakery), que generaliza Dekker/Peterson (2 procesos) a N procesos.

¿Por qué fue relevante?

- Dekker y Peterson solo resolvían el problema para 2 procesos.
- Lamport ofreció la primera solución conocida para N procesos usando solo lectura/escritura simples.
- Introdujo la idea de "orden justo" (FIFO) mediante tickets, evitando la inanición.

¿Qué problema resuelve?

El problema de la sección crítica: cuando N procesos concurrentes comparten un recurso (una variable, un archivo, un dispositivo), es necesario garantizar que solo uno a la vez pueda ejecutar el bloque de código que lo modifica.

***Analogía de la panadería:** cada cliente (proceso) toma un número al entrar; se atiende primero a quien tiene el número más bajo, igual que en la sección crítica.*

1

Exclusión mutua

Nunca dos procesos dentro de la sección crítica al mismo tiempo.

2

Progreso

La decisión de quién entra no se pospone indefinidamente sin razón.

3

Espera limitada

Ningún proceso espera para siempre: existe un límite de turnos que lo adelantan.

Variables compartidas

`choosing[i]`

boolean, tamaño N

Booleano que indica si el proceso i está eligiendo su número. Evita que otro proceso lea el número mientras se está actualizando

`number[i]`

entero, tamaño N

El "ticket" del proceso i : $1 +$ el mayor número visto. Vale 0 si no compete por la sección crítica

Alcance y valores: Ambos arreglos son globales, compartidos por los N procesos. Se inicializan en $\text{choosing}[i] = \text{false}$ y $\text{number}[i] = 0$ para todo i

Propósito conjunto: Juntas forman un sistema de "turnos" (ticket, id) que cualquier proceso puede leer para saber si le toca entrar, sin necesitar instrucciones atómicas especiales de hardware

Ejemplo con 3 procesos

Tres procesos piden turno casi al mismo tiempo. Así quedan sus tickets:

Proceso	Pide turno	number[i] asignado	Orden de entrada a SC
P0	t = 0.0	1	1º
P2	t = 0.3	2	2º
P1	t = 0.6	3	3º

Idea clave: Aunque los tres pidieron turno casi juntos, el ticket más bajo (P0) entra primero, luego P2 y al final P1 — el orden de llegada casi no importa, lo que manda es el número (y el id en caso de empate).

Lógica paso a paso

```
// --- Sección de entrada (proceso i) ---
choosing[i] = true
number[i] = 1 + max(number[0..N-1])
choosing[i] = false

for j = 0 to N-1:
    while choosing[j]:
        skip          // espera a que j elija
    while number[j] != 0 and
        (number[j], j) < (number[i], i):
        skip          // espera su turno

// --- Sección crítica ---
...

// --- Sección de salida ---
number[i] = 0
```

1

Elegir número

El proceso levanta `choosing[i]`, calcula un ticket mayor que todos los existentes y lo baja.

2

Comparar con todos

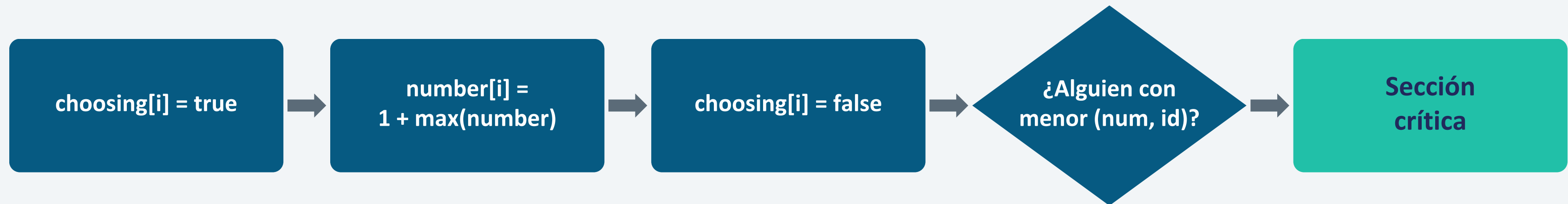
Recorre a los demás procesos `j` y espera si `j` tiene prioridad: menor (`number`, `id`).

3

Entrar y salir

Ejecuta la sección crítica y al salir pone `number[i] = 0`, liberando su turno.

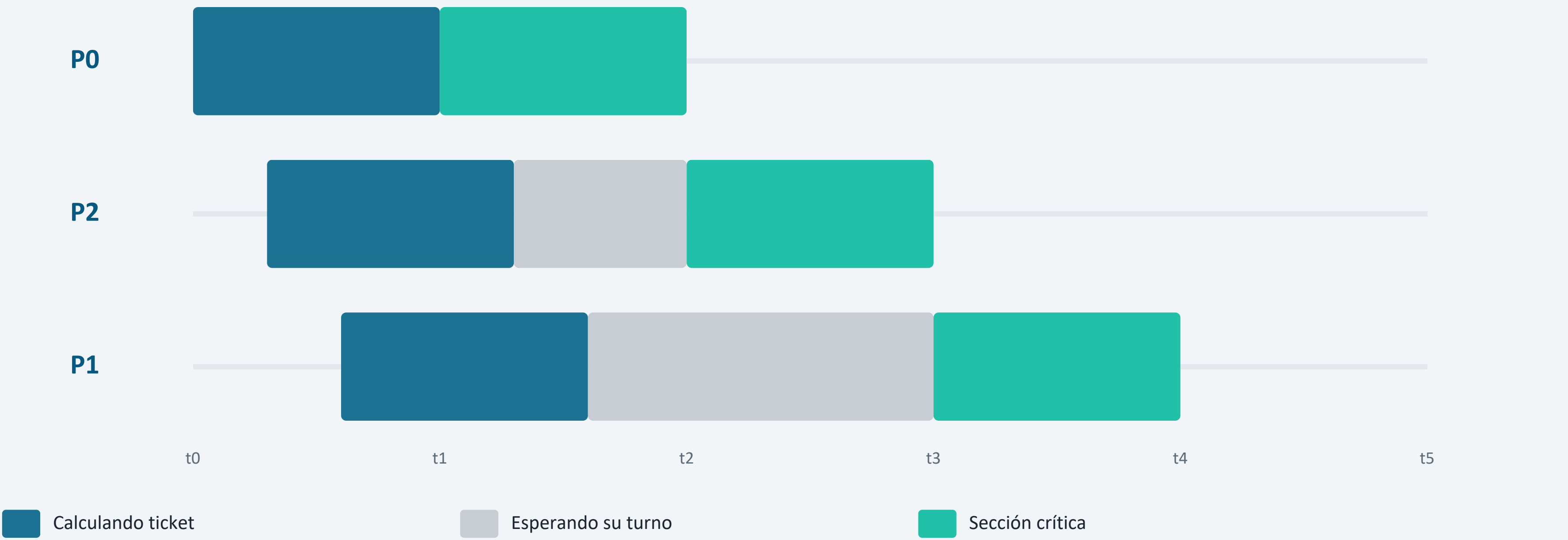
Diagrama de flujo y exclusión mutua



Exclusión mutua:

- Cada proceso obtiene un ticket: el par $(\text{number}[i], i)$.
- Antes de entrar, compara su par contra el de todos los demás en orden lexicográfico: gana el número más bajo; en empate, gana el id más bajo.
- Ese orden es total y estricto $\rightarrow$ nunca dos procesos tienen la prioridad más alta al mismo tiempo $\rightarrow$ exclusión mutua.
- Como respeta quién pidió turno primero, nadie es rebasado para siempre $\rightarrow$ espera limitada.

Traza de ejecución (línea de tiempo)



Simulación con hilos

```
import threading, time
N = 5
choosing = [False]*N
number = [0]*N

def lock(i):
    choosing[i] = True
    number[i] = 1 + max(number)
    choosing[i] = False
    for j in range(N):
        while choosing[j]:
            pass
        while number[j] != 0 and \
            (number[j], j) < (number[i], i):
            pass

def unlock(i):
    number[i] = 0

def proceso(i):
    lock(i)
    print(f"Proceso {i} en SC")
    time.sleep(0.05)
    unlock(i)

hilos = [threading.Thread(target=proceso,
                          args=(i,)) for i in range(N)]
[h.start() for h in hilos]
[h.join() for h in hilos]
```

Orden de tickets

Los prints salen en el orden en que se asignaron los números, no al azar.

Sin condiciones de carrera

Ningún proceso imprime "en sección crítica" mientras otro sigue dentro.

Prueba de estrés

Aumentar N o quitar time.sleep para forzar más contención y verificar que se mantiene el orden.

Salida esperada en consola

```
$ python bakery.py
Proceso 0 en SC
Proceso 2 en SC
Proceso 1 en SC
Proceso 3 en SC
Proceso 4 en SC
$
```

Qué verificar en vivo

- Las líneas nunca se entrelazan: un proceso termina e imprime antes de que el siguiente entre.
- El orden refleja el ticket asignado, no el orden en que se crearon los hilos.
- Si se corre varias veces, el orden puede variar según el sistema operativo, pero nunca se solapan dos procesos en SC.

Ventajas y limitaciones

Ventajas

- Funciona para N procesos, no solo 2.
- No requiere instrucciones atómicas de hardware (TAS/CAS); solo lectura/escritura simples.
- Es justo: respeta el orden de llegada (FIFO por ticket).
- Garantiza espera limitada — ningún proceso muere de inanición.

Limitaciones

- Espera activa (busy-waiting): consume CPU mientras un proceso espera su turno.
- Overhead de memoria y de recorrido $O(N)$ por cada intento de entrada.
- Los tickets crecen sin límite superior fijo (en la práctica se manejan con cuidado o se reinician).
- Más lento que TAS/CAS bajo alta contención, al no usar soporte de hardware.

Comparación con otros algoritmos

Algoritmo	Procesos	Hardware especial	Espera activa	Recomendado cuando...
Dekker	2	No	Sí	Solo 2 procesos, sin hardware atómico
Peterson	2	No	Sí	Solo 2 procesos, solución simple y clásica
Lamport	N	No	Sí	N procesos, se requiere justicia (FIFO) y no hay CAS/TAS disponibles
Test-and-Set	N	Sí (TAS)	Sí	Hay soporte de hardware y la simplicidad importa más que la justicia
CAS	N	Sí (CAS)	Sí (o lock-free)	Alta contención y máximo rendimiento; hardware moderno disponible

En resumen

1

Resuelve exclusión mutua para N procesos

sin necesitar instrucciones atómicas de hardware.

2

Se basa en tickets (number, id)

comparados en orden lexicográfico, como una fila de panadería.

3

Cumple los 3 requisitos clásicos

exclusión mutua, progreso y espera limitada (fairness).

4

Su costo es la espera activa

en sistemas reales con hardware disponible, TAS/CAS suelen preferirse.

